Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе битовых полей»

Выполнил:
студент группы 3825Б1ФИсп1
Шаров Н.С.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 7
3.1. Описание структуры программы 7
3.2. Описание структур данных 7
3.2.1. Класс TBitField 7
3.2.2. Класс TSet 10
3.3. Описание алгоритмов 12
3.3.1. Доступ к отдельному биту 12
3.3.2. Теоретико-множественные операции 12
3.3.3. Решето Эратосфена 13
4. Результаты экспериментов и тестирования 14
Заключение 18
Литература 19
Приложение 20

Введение

Множество — одно из базовых понятий математики и программирования. Во многих задачах (поиск простых чисел, учёт состояний объектов, работа с флагами, графы) приходится хранить подмножества фиксированного конечного универса {0, 1, …, N−1} и быстро выполнять над ними теоретико-множественные операции: объединение, пересечение, дополнение, проверку принадлежности элемента.

Одним из самых экономных способов представления таких множеств является **битовое поле** — последовательность битов, в которой i-й бит равен 1, если элемент i принадлежит множеству, и 0 — если не принадлежит (характеристический вектор множества). На один элемент универса тратится всего один бит памяти, а операции над множествами сводятся к побитовым операциям процессора «ИЛИ», «И» и «НЕ», которые обрабатывают сразу по 32 элемента за одну команду.

В данной работе реализованы класс битового поля `TBitField` и класс множества `TSet`, построенный на его основе, а также набор автоматических тестов на фреймворке Google Test и пример использования — программа поиска простых чисел методом «решета Эратосфена».

1. Постановка задачи

Цель работы — разработка структуры данных для хранения множеств с использованием битовых полей, а также освоение фреймворка для разработки автоматических тестов Google Test.

Для достижения цели необходимо решить следующие задачи:

- реализовать класс битового поля `TBitField` согласно заданному интерфейсу (файл `tbitfield.h`);
- реализовать класс множества `TSet` согласно заданному интерфейсу (файл `tset.h`);
- написать и обеспечить работоспособность тестов, покрывающих основной функционал и краевые случаи.

Объявления классов должны оставаться неизменными, реализация может быть вынесена в отдельные файлы `.cpp`.

Дополнительные требования к реализации:

- при обращении к биту с номером вне диапазона [0, BitLen) и при создании поля отрицательной длины должно генерироваться исключение;
- операции объединения и пересечения множеств допустимы только для множеств из одного универса, то есть с одинаковой мощностью универса `MaxPower`; иначе генерируется исключение;
- операции с элементом (`InsElem`, `DelElem`, `IsMember`, `+ Elem`, `- Elem`) допустимы только для элементов универса; иначе генерируется исключение.

2. Руководство пользователя

Проект собирается системой CMake (файл `CMakeLists.txt`). Библиотека Google Test загружается автоматически при первой сборке. В результате сборки получаются две программы: `sample` — пример использования множества и `tests` — набор автоматических тестов.

Сборка в CLion. Откройте папку `Lab1` командой File → Open, дождитесь окончания загрузки CMake-проекта (вкладка CMake), затем выберите в списке конфигураций вверху окна `sample` или `tests` и нажмите кнопку запуска.

Сборка из командной строки:

```
cmake -S . -B build
cmake --build build
./build/sample
./build/tests
```

Программа sample (решето Эратосфена). После запуска программа выводит приглашение и ожидает ввода целого числа n ≥ 2 — верхней границы поиска. Затем она печатает множество найденных простых чисел, список простых чисел по 10 в строке и их количество. При вводе не числа или числа меньше 2 программа завершается с кодом 1. Пример работы для n = 50 приведён на рис. 1.

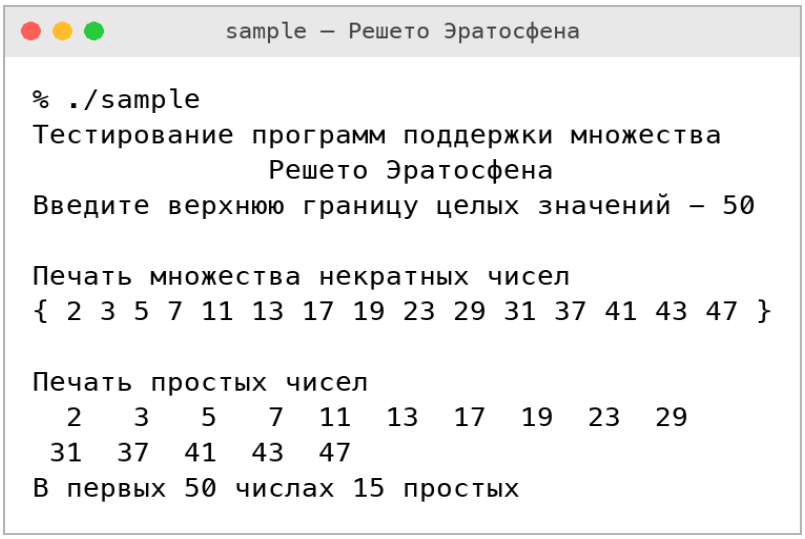


Рис 1. Запуск программы sample для n = 50

Использование классов в своей программе. Достаточно подключить заголовок `tset.h` (или `tbitfield.h`) и собрать программу вместе с файлами из папки `src`. Пример:

```
#include "tset.h"

TSet a(10), b(10);   // два пустых множества из универса {0..9}
a.InsElem(1); a.InsElem(4);   // a = {1, 4}
b = b + 4 + 7;                // b = {4, 7}
TSet u = a + b;      // объединение:  { 1 4 7 }
TSet i = a * b;      // пересечение:  { 4 }
TSet c = ~a;         // дополнение:   { 0 2 3 5 6 7 8 9 }
std::cout << u << std::endl;
```

Форматы ввода и вывода. Битовое поле выводится строкой из символов 0 и 1, где i-й символ — значение i-го бита (например, `01001`); при вводе читается BitLen символов, символ `1` устанавливает бит, любой другой — сбрасывает. Множество выводится в виде списка элементов в фигурных скобках: `{ 1 4 8 }`; при вводе сначала указывается количество элементов, затем сами элементы: `3 1 4 8`.

Программа tests. Запускает все 91 автоматический тест и выводит результат каждого теста (OK — тест пройден, FAILED — не пройден) и итоговую сводку. Пример вывода приведён на рис. 2.

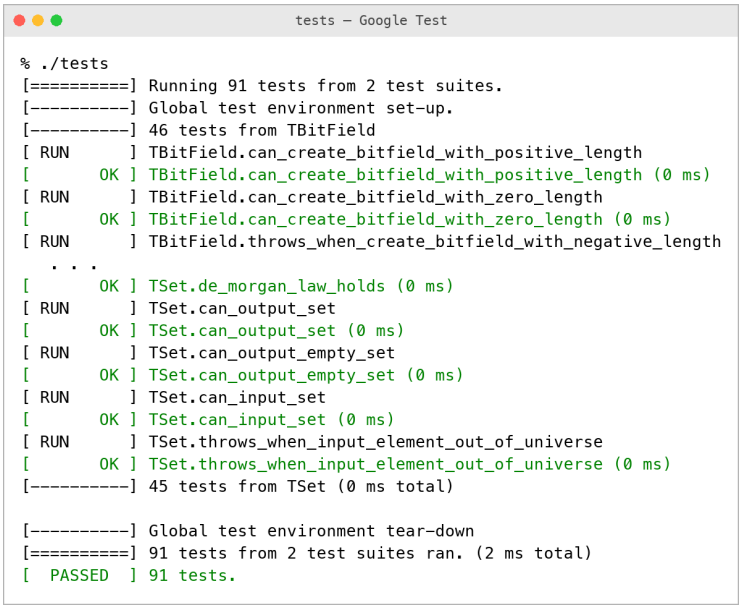


Рис 2. Результат запуска тестов

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из следующих модулей:

Таблица 1. Состав модулей программы

Файл	Назначение
include/tbitfield.h	Объявление класса битового поля `TBitField` и типа `TELEM` (интерфейс задан преподавателем).
include/tset.h	Объявление класса множества `TSet` (интерфейс задан преподавателем).
src/tbitfield.cpp	Реализация методов класса `TBitField` и операторов ввода/вывода битового поля.
src/tset.cpp	Реализация методов класса `TSet` и операторов ввода/вывода множества.
samples/sample_prime_numbers.cpp	Пример использования: поиск простых чисел методом решета Эратосфена (функция `main`).
test/test_main.cpp	Точка входа тестовой программы: инициализация и запуск Google Test.
test/test_tbitfield.cpp	46 тестов класса `TBitField`.
test/test_tset.cpp	45 тестов класса `TSet`.
CMakeLists.txt	Описание сборки: загрузка Google Test, библиотека `set` из файлов `src`, программы `sample` и `tests`.

Файлы `src/tbitfield.cpp` и `src/tset.cpp` собираются в статическую библиотеку `set`, которую используют обе программы — `sample` и `tests`. Таким образом классы компилируются один раз, а заголовки из папки `include` доступны всем модулям, подключающим библиотеку.

3.2. Описание структур данных

Тип `TELEM` — псевдоним (`typedef`) для `unsigned int`. Значение этого типа (32 бита) используется как элемент памяти, хранящий 32 бита битового поля. Беззнаковый тип выбран для того, чтобы все 32 разряда были равноправными и операции сдвига работали без особенностей знакового бита.

Всюду далее n — длина битового поля (`BitLen`), m = ⌈n/32⌉ — число элементов памяти (`MemLen`), P — мощность универса множества (`MaxPower`).

3.2.1. *Класс TBitField*

Класс представляет битовое поле — последовательность из `BitLen` битов с номерами от 0 до BitLen−1. Поля класса (закрытые):

- `int BitLen` — длина битового поля (количество битов);
- `TELEM *pMem` — указатель на динамический массив элементов памяти, в которых хранятся биты;
- `int MemLen` — количество элементов в массиве `pMem`, MemLen = ⌈BitLen/32⌉.

Бит с номером k хранится в элементе `pMem[k / 32]` в разряде `k % 32`; биты внутри элемента нумеруются справа налево (от младших разрядов к старшим). Неиспользуемые старшие разряды последнего элемента всегда равны нулю — это инвариант класса, на котором основано сравнение полей поэлементно.

Ниже описаны все методы и функции класса.

`TBitField(int len)`

Параметры: `len` — длина битового поля.

Описание: конструктор. Проверяет, что len ≥ 0 (иначе генерирует `invalid_argument`), вычисляет MemLen = (len + 31) / 32 (деление с округлением вверх), выделяет массив `pMem` и обнуляет его — созданное поле пусто.

Сложность: O(m): выделение памяти выполняется за O(1), цикл обнуления выполняет m итераций.

`TBitField(const TBitField &bf)`

Параметры: `bf` — копируемое битовое поле.

Описание: конструктор копирования. Копирует длины, выделяет собственный массив памяти и переписывает в него элементы `bf`. Благодаря отдельной памяти копия и оригинал не влияют друг на друга.

Сложность: O(m): цикл копирования выполняет m итераций.

`~TBitField()`

Описание: деструктор. Освобождает память, выделенную под массив `pMem`.

Сложность: O(1): `delete[]` для массива встроенного типа не вызывает деструкторы элементов.

`int GetMemIndex(const int n) const`

Параметры: `n` — номер бита.

Возвращаемое значение: индекс элемента массива `pMem`, содержащего бит n.

Описание: закрытый вспомогательный метод, вычисляет `n / 32`.

Сложность: O(1).

`TELEM GetMemMask(const int n) const`

Параметры: `n` — номер бита.

Возвращаемое значение: битовая маска — значение `TELEM`, в котором единица стоит только в разряде, соответствующем биту n.

Описание: закрытый вспомогательный метод, вычисляет `1u << (n % 32)`.

Сложность: O(1).

`int GetLength(void) const`

Возвращаемое значение: длина битового поля `BitLen`.

Описание: возвращает количество битов поля.

Сложность: O(1).

`void SetBit(const int n)`

Параметры: `n` — номер бита.

Описание: устанавливает бит n в 1: `pMem[GetMemIndex(n)] |= GetMemMask(n)`. Остальные биты не изменяются. Если n < 0 или n ≥ BitLen, генерируется `out_of_range`.

Сложность: O(1).

`void ClrBit(const int n)`

Параметры: `n` — номер бита.

Описание: сбрасывает бит n в 0: `pMem[GetMemIndex(n)] &= ~GetMemMask(n)`. Остальные биты не изменяются. При неверном номере генерируется `out_of_range`.

Сложность: O(1).

`int GetBit(const int n) const`

Параметры: `n` — номер бита.

Возвращаемое значение: 1, если бит установлен, иначе 0.

Описание: проверяет бит n, накладывая маску операцией «И». При неверном номере генерируется `out_of_range`.

Сложность: O(1).

`TBitField& operator=(const TBitField &bf)`

Параметры: `bf` — присваиваемое битовое поле.

Возвращаемое значение: ссылка на текущий объект (для цепочек присваиваний `a = b = c`).

Описание: присваивание. При самоприсваивании ничего не делает. Иначе освобождает старый массив, выделяет новый размера `bf.MemLen` и копирует в него элементы `bf`.

Сложность: O(m_bf), где m_bf — число элементов памяти поля `bf`: цикл копирования.

`int operator==(const TBitField &bf) const`

Параметры: `bf` — сравниваемое поле.

Возвращаемое значение: 1, если поля равны, иначе 0.

Описание: поля равны, если совпадают их длины и все элементы памяти. Поэлементное сравнение корректно благодаря инварианту о нулевых неиспользуемых разрядах.

Сложность: O(m) в худшем случае: цикл сравнения проходит не более m элементов.

`int operator!=(const TBitField &bf) const`

Параметры: `bf` — сравниваемое поле.

Возвращаемое значение: 1, если поля различны, иначе 0.

Описание: возвращает отрицание результата `operator==`.

Сложность: O(m) — как у `operator==`.

`TBitField operator|(const TBitField &bf)`

Параметры: `bf` — второй операнд.

Возвращаемое значение: новое битовое поле — результат операции «ИЛИ».

Описание: длина результата равна большей из длин операндов. Результат создаётся пустым, в него копируются элементы текущего поля, затем на них накладываются элементы `bf` операцией «ИЛИ». Операнды не изменяются.

Сложность: $O(\max(m_1, m_2))$: создание результата — $O(\max(m_1, m_2))$, два цикла — m_1 и m_2 итераций.

`TBitField operator&(const TBitField &bf)`

Параметры: `bf` — второй операнд.

Возвращаемое значение: новое битовое поле — результат операции «И».

Описание: длина результата равна большей из длин. Операция выполняется только для общих элементов памяти ($\min(m_1, m_2)$); остальные элементы результата остаются нулевыми, так как в коротком операнде соответствующих битов нет.

Сложность: $O(\max(m_1, m_2))$: обнуление результата — $O(\max(m_1, m_2))$, цикл — $\min(m_1, m_2)$ итераций.

`TBitField operator~(void)`

Возвращаемое значение: новое битовое поле — отрицание текущего.

Описание: создаёт пустое поле той же длины и устанавливает в нём каждый бит, равный 0 в исходном поле. Перебор ведётся по битам, а не по элементам памяти, чтобы не установить неиспользуемые разряды последнего элемента и сохранить инвариант класса.

Сложность: O(n): цикл выполняет n итераций, в каждой — `GetBit` и, возможно, `SetBit` за O(1); создание результата — $O(m) \le O(n)$.

`istream& operator>>(istream &istr, TBitField &bf)`

Параметры: `istr` — поток ввода, `bf` — вводимое поле.

Возвращаемое значение: ссылка на поток `istr` (для цепочек ввода).

Описание: дружественная функция. Читает BitLen символов; символ `1` устанавливает соответствующий бит, любой другой символ сбрасывает его.

Сложность: O(n): по одной операции чтения и установки бита на каждый из n битов.

`ostream& operator<<(ostream &ostr, const TBitField &bf)`

Параметры: `ostr` — поток вывода, `bf` — выводимое поле.

Возвращаемое значение: ссылка на поток `ostr`.

Описание: дружественная функция. Выводит значения всех битов подряд символами 0 и 1.

Сложность: O(n): по одному вызову `GetBit` на каждый бит.

3.2.2. *Класс TSet*

Класс представляет множество — подмножество универса {0, 1, …, MaxPower−1}. Поля класса (закрытые):

- `int MaxPower` — мощность универса (максимальная мощность множества);
- `TBitField BitField` — битовое поле длины `MaxPower`, характеристический вектор множества: бит i равен 1 тогда и только тогда, когда элемент i принадлежит множеству.

Класс `TSet` не работает с памятью напрямую: всё управление памятью выполняет вложенный объект `TBitField`, поэтому собственный деструктор классу не нужен. Большинство методов делегируют работу соответствующим методам битового поля. Обозначим p = ⌈P/32⌉.

`TSet(int mp)`

Параметры: `mp` — мощность универса.

Описание: конструктор. В списке инициализации задаёт `MaxPower = mp` и создаёт битовое поле длины `mp`; множество пусто. При `mp < 0` исключение генерирует конструктор `TBitField`.

Сложность: O(p) — сложность конструктора `TBitField`.

`TSet(const TSet &s)`

Параметры: `s` — копируемое множество.

Описание: конструктор копирования, копирует мощность и битовое поле.

Сложность: O(p).

`TSet(const TBitField &bf)`

Параметры: `bf` — битовое поле.

Описание: конструктор преобразования типа: создаёт множество, характеристическим вектором которого является `bf`; мощность универса равна длине поля.

Сложность: O(p) — копирование поля.

`operator TBitField()`

Возвращаемое значение: копия битового поля множества.

Описание: оператор преобразования множества к битовому полю.

Сложность: O(p) — копирование поля.

`int GetMaxPower(void) const`

Возвращаемое значение: мощность универса `MaxPower`.

Описание: возвращает мощность универса.

Сложность: O(1).

`void InsElem(const int Elem)`

Параметры: `Elem` — добавляемый элемент.

Описание: включает элемент в множество (устанавливает бит `Elem`). Для элемента вне универса генерируется `out_of_range`.

Сложность: O(1).

`void DelElem(const int Elem)`

Параметры: `Elem` — удаляемый элемент.

Описание: исключает элемент из множества (сбрасывает бит `Elem`). Для элемента вне универса генерируется `out_of_range`.

Сложность: O(1).

`int IsMember(const int Elem) const`

Параметры: `Elem` — проверяемый элемент.

Возвращаемое значение: 1, если элемент принадлежит множеству, иначе 0.

Описание: проверяет бит `Elem`. Для элемента вне универса генерируется `out_of_range`.

Сложность: O(1).

`TSet& operator=(const TSet &s)`

Параметры: `s` — присваиваемое множество.

Возвращаемое значение: ссылка на текущий объект.

Описание: присваивание: копирует мощность универса и битовое поле (через `TBitField::operator=`).

Сложность: O(p).

`int operator==(const TSet &s) const`

Параметры: `s` — сравниваемое множество.

Возвращаемое значение: 1, если множества равны, иначе 0.

Описание: множества равны, если совпадают мощности универсов и характеристические векторы.

Сложность: O(p).

`int operator!=(const TSet &s) const`

Параметры: `s` — сравниваемое множество.

Возвращаемое значение: 1, если множества различны, иначе 0.

Описание: отрицание `operator==`.

Сложность: O(p).

`TSet operator+(const TSet &s)`

Параметры: `s` — второе множество.

Возвращаемое значение: объединение множеств.

Описание: если мощности универсов различны, генерируется `invalid_argument` (множества из разных универсов). Иначе возвращается множество, построенное по полю `BitField | s.BitField`.

Сложность: O(p) — операция «ИЛИ» над полями и создание результата.

`TSet operator*(const TSet &s)`

Параметры: `s` — второе множество.

Возвращаемое значение: пересечение множеств.

Описание: если мощности универсов различны, генерируется `invalid_argument`. Иначе возвращается множество, построенное по полю `BitField & s.BitField`.

Сложность: O(p).

`TSet operator~(void)`

Возвращаемое значение: дополнение множества до универса.

Описание: возвращает множество, построенное по полю `~BitField`.

Сложность: O(P) — сложность отрицания битового поля длины P.

`TSet operator+(const int Elem)`

Параметры: `Elem` — добавляемый элемент.

Возвращаемое значение: новое множество, равное текущему с добавленным элементом.

Описание: создаёт копию множества и включает в неё элемент; исходное множество не изменяется. Для элемента вне универса генерируется `out_of_range`.

Сложность: O(p) — копирование множества.

`TSet operator-(const int Elem)`

Параметры: `Elem` — удаляемый элемент.

Возвращаемое значение: новое множество, равное текущему без элемента.

Описание: создаёт копию множества и исключает из неё элемент; исходное множество не изменяется.

Сложность: O(p).

`istream& operator>>(istream &istr, TSet &s)`

Параметры: `istr` — поток ввода, `s` — вводимое множество.

Возвращаемое значение: ссылка на поток `istr`.

Описание: дружественная функция. Читает количество элементов k, затем k элементов и включает каждый в множество. Элемент вне универса приводит к исключению `out_of_range`.

Сложность: O(k) — k операций чтения и включения элемента за O(1).

`ostream& operator<<(ostream &ostr, const TSet &s)`

Параметры: `ostr` — поток вывода, `s` — выводимое множество.

Возвращаемое значение: ссылка на поток `ostr`.

Описание: дружественная функция. Перебирает элементы универса 0…P−1 и выводит принадлежащие множеству в виде `{ a b c }`.

Сложность: O(P) — по одной проверке `IsMember` на каждый элемент универса.

3.3. Описание алгоритмов

3.3.1. *Доступ к отдельному биту*

Для бита с номером k вычисляются индекс элемента памяти `k / 32` и маска `1u << (k % 32)` — число, в котором единица стоит только в нужном разряде. Затем:

- установка бита — операция «ИЛИ» элемента с маской: нужный разряд становится 1, остальные не меняются;
- сброс бита — операция «И» элемента с инвертированной маской `~маска`: нужный разряд становится 0;
- проверка бита — операция «И» элемента с маской: результат отличен от нуля тогда и только тогда, когда бит установлен.

Например, для k = 40 получаем элемент `pMem[1]` и маску `1u << 8`. Все три операции выполняются за константное время O(1).

3.3.2. *Теоретико-множественные операции*

Принадлежность элемента объединению A ∪ B означает принадлежность A **или** B, пересечению A ∩ B — принадлежность A **и** B, дополнению — **не**принадлежность A. Поэтому операции над множествами реализуются побитовыми операциями над характеристическими векторами:

Таблица 2. Соответствие операций над множествами и битовых операций

Операция	Запись	Битовая операция	Сложность
Объединение	`A + B`	`|` по элементам памяти	O(P/32)
Пересечение	`A * B`	`&` по элементам памяти	O(P/32)
Дополнение	`~A`	`~` по битам	O(P)
Добавление элемента	`A + e`	`SetBit(e)` в копии	O(P/32)
Удаление элемента	`A - e`	`ClrBit(e)` в копии	O(P/32)
Принадлежность	`IsMember(e)`	`GetBit(e)`	O(1)

Объединение и пересечение обрабатывают по 32 элемента универса за одну машинную операцию, поэтому, например, объединение двух множеств из универса мощностью 1000 требует всего $\lceil 1000/32 \rceil = 32$ операции «ИЛИ».

Отрицание битового поля выполняется по отдельным битам. Если инвертировать элементы памяти целиком, единицы появились бы и в неиспользуемых разрядах последнего элемента (например, у поля длины 5 используются только 5 младших разрядов из 32), что нарушило бы корректность поэлементного сравнения.

3.3.3. *Решето Эратосфена*

Программа `sample` находит все простые числа, не превосходящие n:

- создаётся множество с универсом $\{0, \ldots, n\}$, в него включаются все числа от 2 до n;
- для каждого $m = 2, 3, \ldots$ пока $m^2 \le n$: если m ещё принадлежит множеству, то m — простое, и из множества исключаются все кратные ему числа 2m, 3m, …, не превосходящие n;
- оставшиеся в множестве числа — простые.

Проверку достаточно вести до $\sqrt{n}$: у любого составного числа $k \le n$ есть делитель, не превосходящий $\sqrt{k} \le \sqrt{n}$, поэтому к этому моменту все составные числа уже исключены.

Оценка сложности. Заполнение множества и печать результата требуют O(n) операций. При вычёркивании для каждого простого $p \le \sqrt{n}$ выполняется не более n/p операций `DelElem` за O(1). Суммарно $n \cdot (1/2 + 1/3 + 1/5 + \ldots) = n \cdot \Sigma_{p \le \sqrt{n}} \, 1/p = O(n \log \log n)$, так как сумма обратных простых чисел, не превосходящих x, растёт как $\ln \ln x$. Итоговая сложность — **$O(n \log \log n)$**, память — $n + 1$ бит.

4. Результаты экспериментов и тестирования

Для проверки корректности разработано 91 автоматический тест на фреймворке Google Test: 46 тестов класса `TBitField` и 45 тестов класса `TSet`. Все тесты написаны самостоятельно. Каждый тест подготавливает данные, выполняет операцию и проверяет результат макросами `EXPECT_EQ`, `EXPECT_NE`, `EXPECT_TRUE`, `EXPECT_FALSE`, а генерацию исключений — макросами `ASSERT_ANY_THROW` и `ASSERT_NO_THROW`. Кроме основного функционала, проверяются краевые случаи:

- поле и универс нулевой длины, отрицательная длина;
- первый и последний бит поля, биты 31 и 32 на границе соседних элементов памяти;
- отрицательные и слишком большие номера битов и элементов;
- самоприсваивание, цепочка присваиваний, независимость памяти копии и оригинала;
- операции над полями разной длины и множествами разной мощности;
- отсутствие «лишних» единиц в неиспользуемых разрядах после отрицания;
- законы теории множеств: $A \cup \neg A = U$, $A \cap \neg A = \emptyset$, законы де Моргана.

Краткие комментарии ко всем тестам приведены в таблицах ниже.

Таблица 3. Тесты класса TBitField

№	Тест	Что проверяет
1	can_create_bitfield_with_positive_length	создание поля положительной длины не генерирует исключений
2	can_create_bitfield_with_zero_length	можно создать поле нулевой длины, его длина равна 0
3	throws_when_create_bitfield_with_negative_length	при отрицательной длине генерируется исключение
4	can_get_length	`GetLength` возвращает длину, заданную при создании
5	new_bitfield_is_set_to_zero	все биты нового поля из 100 битов равны 0
6	can_set_bit	после `SetBit` бит становится равным 1
7	setting_bit_does_not_change_other_bits	установка бита 33 не меняет остальные 63 бита
8	can_clear_bit	после `ClrBit` установленный бит становится равным 0
9	can_set_and_clear_first_and_last_bits	работа с первым (0) и последним (64) битами поля из 65 битов
10	can_set_bits_on_element_boundary	биты 31 и 32 из разных элементов памяти не влияют на соседей
11	throws_when_set_bit_with_negative_index	`SetBit` с отрицательным индексом генерирует исключение
12	throws_when_set_bit_with_too_large_index	`SetBit(BitLen)` генерирует исключение
13	throws_when_get_bit_with_negative_index	`GetBit` с отрицательным индексом генерирует исключение
14	throws_when_get_bit_with_too_large_index	`GetBit` с индексом больше длины генерирует исключение
15	throws_when_clear_bit_with_negative_index	`ClrBit` с отрицательным индексом генерирует исключение
16	throws_when_clear_bit_with_too_large_index	`ClrBit` с индексом больше длины генерирует исключение

№	Тест	Что проверяет
17	throws_on_any_bit_access_in_zero_length_bitfield	любое обращение к биту поля нулевой длины — исключение
18	can_create_copied_bitfield	конструктор копирования не генерирует исключений
19	copied_bitfield_is_equal_to_source_one	копия равна оригиналу
20	copied_bitfield_has_its_own_memory	изменение копии не меняет оригинал
21	can_assign_bitfields_of_equal_size	присваивание полей одинаковой длины копирует биты
22	assign_operator_changes_bitfield_size	после присваивания длина равна длине правого операнда
23	can_assign_bitfields_of_non_equal_size	присваивание поля большей длины (2 ← 100) работает корректно
24	can_assign_to_itself	самоприсваивание не портит поле
25	assigned_bitfield_has_its_own_memory	после присваивания поля не разделяют память
26	compare_equal_bitfields_of_equal_size	равные поля одинаковой длины равны
27	compare_non_equal_bitfields_of_equal_size	поля с разными битами не равны
28	bitfields_of_non_equal_size_are_not_equal	поля разной длины не равны
29	empty_bitfields_are_equal	два поля нулевой длины равны
30	or_operator_applied_to_bitfields_of_equal_size	«ИЛИ» полей одинаковой длины: 0011 | 0101 = 0111
31	or_operator_applied_to_bitfields_of_non_equal_size	«ИЛИ» полей длины 4 и 5 в обоих порядках, длина результата 5
32	or_operator_does_not_change_operands	операция «ИЛИ» не изменяет операнды
33	and_operator_applied_to_bitfields_of_equal_size	«И» полей одинаковой длины: 0011 & 0101 = 0001
34	and_operator_applied_to_bitfields_of_non_equal_size	«И» полей длины 4 и 5 в обоих порядках
35	and_operator_applied_to_large_bitfields_of_non_equal_size	«И» полей длины 40 и 100 (разное число элементов памяти)
36	can_invert_bitfield	отрицание поля из 2 битов
37	can_invert_large_bitfield	отрицание поля из 38 битов (два элемента памяти)
38	invert_does_not_set_bits_outside_of_length	отрицание пустого поля длины 5 равно полю из пяти единиц
39	double_invert_gives_source_bitfield	двойное отрицание даёт исходное поле
40	invert_plus_and_operator_on_different_size_bitfield	комбинация отрицания и «И» для полей длины 4 и 8
41	can_invert_zero_length_bitfield	отрицание поля нулевой длины
42	can_invert_many_random_bits_bitfield	отрицание поля с несколькими произвольными битами
43	bitfields_with_different_bits_are_not_equal	поля длины 38 с разными битами не равны
44	can_output_bitfield	вывод поля в поток: `01001`
45	can_input_bitfield	ввод поля из строки `10010`
46	output_then_input_gives_same_bitfield	вывод и последующий ввод дают исходное поле

Таблица 4. Тесты класса TSet

№	Тест	Что проверяет
1	can_get_max_power_set	`GetMaxPower` возвращает мощность, заданную при создании
2	throws_when_create_set_with_negative_power	отрицательная мощность — исключение
3	can_create_empty_universe_set	можно создать множество с универсом мощности 0
4	new_set_is_empty	новое множество не содержит элементов
5	can_insert_non_existing_element	добавление отсутствующего элемента
6	can_insert_existing_element	повторное добавление элемента не нарушает множество
7	can_delete_non_existing_element	удаление отсутствующего элемента
8	can_delete_existing_element	удаление имеющегося элемента
9	throws_when_insert_element_out_of_universe	добавление элемента вне универса — исключение
10	throws_when_delete_element_out_of_universe	удаление элемента вне универса — исключение
11	throws_when_check_membership_out_of_universe	проверка элемента вне универса — исключение
12	copied_set_is_equal_to_source_one	копия множества равна оригиналу
13	copied_set_has_its_own_memory	изменение копии не меняет оригинал
14	can_create_set_from_bitfield	создание множества из битового поля
15	can_convert_set_to_bitfield	преобразование множества к битовому полю
16	conversion_to_bitfield_and_back_gives_same_set	преобразование в поле и обратно даёт исходное множество
17	can_assign_set_of_equal_size	присваивание множеств одинаковой мощности
18	can_assign_set_of_non_equal_size	присваивание множества другой мощности меняет мощность
19	can_assign_set_to_itself	самоприсваивание не портит множество
20	can_chain_assignment	цепочка присваиваний `a = b = c`
21	compare_two_sets_of_non_equal_sizes	множества разной мощности не равны
22	compare_two_equal_sets	равные множества равны
23	compare_two_non_equal_sets	множества с разными элементами не равны
24	can_insert_non_existing_element_using_plus_operator	`A + e` добавляет отсутствующий элемент
25	plus_element_does_not_change_source_set	`A + e` не изменяет исходное множество
26	throws_when_insert_non_existing_element_out_of_range_using_plus_operator	`A + e` для элемента вне универса — исключение
27	can_insert_existing_element_using_plus_operator	`A + e` для имеющегося элемента не меняет множество
28	can_delete_element_using_minus_operator	`A - e` удаляет элемент, исходное множество не меняется

№	Тест	Что проверяет
29	can_delete_non_existing_element_using_minus_operator	`A - e` для отсутствующего элемента не меняет множество
30	throws_when_delete_element_out_of_range_using_minus_operator	`A - e` для элемента вне универса — исключение
31	can_combine_two_sets_of_equal_size	объединение {1,2,4} ∪ {0,1,2} = {0,1,2,4}
32	check_size_of_the_combination_of_two_sets_of_equal_size	мощность универса объединения равна мощности операндов
33	throws_when_combine_two_sets_of_non_equal_size	объединение множеств разной мощности — исключение
34	union_with_empty_set_gives_same_set	объединение с пустым множеством не меняет множество
35	can_intersect_two_sets_of_equal_size	пересечение {1,2,4} ∩ {0,1,2} = {1,2}
36	throws_when_intersect_two_sets_of_non_equal_size	пересечение множеств разной мощности — исключение
37	intersection_of_disjoint_sets_is_empty	пересечение непересекающихся множеств пусто
38	check_negation_operator	дополнение {1,3} в универсе {0..3} равно {0,2}
39	complement_of_empty_set_is_universe	дополнение пустого множества — весь универс (40 элементов)
40	union_with_complement_is_universe_and_intersection_is_empty	A ∪ ¬A = U и A ∩ ¬A = ∅ (универс из 37 элементов)
41	de_morgan_law_holds	закон де Моргана: ¬(A ∪ B) = ¬A ∩ ¬B
42	can_output_set	вывод множества: `{ 0 3 9 }`
43	can_output_empty_set	вывод пустого множества: `{ }`
44	can_input_set	ввод множества из строки `3 1 4 8`
45	throws_when_input_element_out_of_universe	ввод элемента вне универса — исключение

Результаты. Все 91 тест успешно пройдены (рис. 2). Дополнительно программа `sample` проверена на n = 10 (найдено 4 простых числа: 2, 3, 5, 7), n = 50 (15 простых чисел) и n = 1000 (168 простых чисел); результаты совпадают с известными значениями. Тесты и программа `sample` также запускались со средствами AddressSanitizer и UndefinedBehaviorSanitizer, которые не выявили ошибок работы с памятью и неопределённого поведения.

Заключение

В ходе выполнения лабораторной работы были получены следующие результаты:

- реализован класс битового поля `TBitField`, хранящий биты в массиве 32-битных элементов и поддерживающий установку, сброс и проверку битов за O(1), а также операции «ИЛИ», «И», «НЕ», сравнение, присваивание, ввод и вывод;
- реализован класс множества `TSet` на основе битового поля с операциями объединения, пересечения, дополнения, добавления и удаления элементов; операции с множествами из разных универсов запрещены;
- корректность обработки ошибок обеспечена генерацией исключений при выходе за границы и недопустимых операциях;
- разработано 91 автоматический тест на Google Test, покрывающих основной функционал и краевые случаи; все тесты пройдены;
- написан пример использования — поиск простых чисел методом решета Эратосфена со сложностью O(n log log n).

Работа позволила освоить представление множеств битовыми полями, побитовые операции языка C++, управление динамической памятью в классах (конструктор копирования, оператор присваивания, деструктор) и разработку автоматических тестов.

Литература

1. Страуструп Б. Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Кормен Т.Х., Лейзерсон Ч.И., Ривест Р.Л., Штайн К. Алгоритмы: построение и анализ. – 3-е изд. – М.: Вильямс, 2013. – 1328 с.
3. Сайт cppreference: https://en.cppreference.com/
4. Документация GoogleTest: https://google.github.io/googletest/

Приложение

Ниже приведены исходные тексты реализации классов.

src/tbitfield.cpp

```
#include "tbitfield.h"

#include <stdexcept>

using namespace std;

const int BITS_IN_ELEM = sizeof(TELEM) * 8;

TBitField::TBitField(int len)
{
  if (len < 0)
    throw invalid_argument("negative length");
  BitLen = len;
  MemLen = (len + BITS_IN_ELEM - 1) / BITS_IN_ELEM;
  pMem = new TELEM[MemLen];
  for (int i = 0; i < MemLen; i++)
    pMem[i] = 0;
}

TBitField::TBitField(const TBitField &bf)
{
  BitLen = bf.BitLen;
  MemLen = bf.MemLen;
  pMem = new TELEM[MemLen];
  for (int i = 0; i < MemLen; i++)
    pMem[i] = bf.pMem[i];
}

TBitField::~TBitField()
{
  delete[] pMem;
}

int TBitField::GetMemIndex(const int n) const
{
  return n / BITS_IN_ELEM;
}

TELEM TBitField::GetMemMask(const int n) const
{
  return 1u << (n % BITS_IN_ELEM);
}

int TBitField::GetLength(void) const
{
  return BitLen;
}

void TBitField::SetBit(const int n)
{
  if (n < 0 || n >= BitLen)
    throw out_of_range("wrong bit index");
  pMem[GetMemIndex(n)] |= GetMemMask(n);
}

void TBitField::ClrBit(const int n)
{
  if (n < 0 || n >= BitLen)
    throw out_of_range("wrong bit index");
  pMem[GetMemIndex(n)] &= ~GetMemMask(n);
}
```

```
int TBitField::GetBit(const int n) const
{
  if (n < 0 || n >= BitLen)
    throw out_of_range("wrong bit index");
  if (pMem[GetMemIndex(n)] & GetMemMask(n))
    return 1;
  return 0;
}

TBitField& TBitField::operator=(const TBitField &bf)
{
  if (this != &bf)
  {
    delete[] pMem;
    BitLen = bf.BitLen;
    MemLen = bf.MemLen;
    pMem = new TELEM[MemLen];
    for (int i = 0; i < MemLen; i++)
      pMem[i] = bf.pMem[i];
  }
  return *this;
}

int TBitField::operator==(const TBitField &bf) const
{
  if (BitLen != bf.BitLen)
    return 0;
  for (int i = 0; i < MemLen; i++)
    if (pMem[i] != bf.pMem[i])
      return 0;
  return 1;
}

int TBitField::operator!=(const TBitField &bf) const
{
  return !(*this == bf);
}

TBitField TBitField::operator|(const TBitField &bf)
{
  int len = BitLen;
  if (bf.BitLen > len)
    len = bf.BitLen;
  TBitField res(len);
  for (int i = 0; i < MemLen; i++)
    res.pMem[i] = pMem[i];
  for (int i = 0; i < bf.MemLen; i++)
    res.pMem[i] |= bf.pMem[i];
  return res;
}

TBitField TBitField::operator&(const TBitField &bf)
{
  int len = BitLen;
  if (bf.BitLen > len)
    len = bf.BitLen;
  int minMemLen = MemLen;
  if (bf.MemLen < minMemLen)
    minMemLen = bf.MemLen;
  TBitField res(len);
  for (int i = 0; i < minMemLen; i++)
    res.pMem[i] = pMem[i] & bf.pMem[i];
  return res;
}

TBitField TBitField::operator~(void)
{
  TBitField res(BitLen);
```

```
  for (int i = 0; i < BitLen; i++)
    if (GetBit(i) == 0)
      res.SetBit(i);
  return res;
}

istream &operator>>(istream &istr, TBitField &bf)
{
  char c;
  for (int i = 0; i < bf.BitLen; i++)
  {
    istr >> c;
    if (c == '1')
      bf.SetBit(i);
    else
      bf.ClrBit(i);
  }
  return istr;
}

ostream &operator<<(ostream &ostr, const TBitField &bf)
{
  for (int i = 0; i < bf.BitLen; i++)
    ostr << bf.GetBit(i);
  return ostr;
}
```

src/tset.cpp

```
#include "tset.h"

#include <stdexcept>

using namespace std;

TSet::TSet(int mp) : MaxPower(mp), BitField(mp)
{
}

TSet::TSet(const TSet &s) : MaxPower(s.MaxPower), BitField(s.BitField)
{
}

TSet::TSet(const TBitField &bf) : MaxPower(bf.GetLength()), BitField(bf)
{
}

TSet::operator TBitField()
{
  return BitField;
}

int TSet::GetMaxPower(void) const
{
  return MaxPower;
}

int TSet::IsMember(const int Elem) const
{
  return BitField.GetBit(Elem);
}

void TSet::InsElem(const int Elem)
{
  BitField.SetBit(Elem);
}

void TSet::DelElem(const int Elem)
```

```
{
  BitField.ClrBit(Elem);
}

TSet& TSet::operator=(const TSet &s)
{
  MaxPower = s.MaxPower;
  BitField = s.BitField;
  return *this;
}

int TSet::operator==(const TSet &s) const
{
  return MaxPower == s.MaxPower && BitField == s.BitField;
}

int TSet::operator!=(const TSet &s) const
{
  return !(*this == s);
}

TSet TSet::operator+(const TSet &s)
{
  if (MaxPower != s.MaxPower)
    throw invalid_argument("sets from different universes");
  return TSet(BitField | s.BitField);
}

TSet TSet::operator+(const int Elem)
{
  TSet res(*this);
  res.InsElem(Elem);
  return res;
}

TSet TSet::operator-(const int Elem)
{
  TSet res(*this);
  res.DelElem(Elem);
  return res;
}

TSet TSet::operator*(const TSet &s)
{
  if (MaxPower != s.MaxPower)
    throw invalid_argument("sets from different universes");
  return TSet(BitField & s.BitField);
}

TSet TSet::operator~(void)
{
  return TSet(~BitField);
}

istream &operator>>(istream &istr, TSet &s)
{
  int count, elem;
  istr >> count;
  for (int i = 0; i < count; i++)
  {
    istr >> elem;
    s.InsElem(elem);
  }
  return istr;
}

ostream& operator<<(ostream &ostr, const TSet &s)
{
  ostr << "{ ";
```

```
  for (int i = 0; i < s.MaxPower; i++)
    if (s.IsMember(i))
      ostr << i << " ";
  ostr << "}";
  return ostr;
}
```